NAME: ESHA D VERMA

USN:24BTRCN007

COUSRE CODE:24BSCSMA21

MATLAB: EXPERIMENT 1

CLASS:CSE-GEN 'A'

1) A retail store owner tracks daily sales revenue (in $100s) for two weeks:

Dataset: 105, 98, 120, 130, 125, 110, 105, 115, 100, 140, 135, 120, 115, 125.

```
 import numpy as np

# Daily sales revenue (in $100s)
sales_revenue = [105, 98, 120, 130, 125, 110, 105, 115, 100, 140, 135, 120, 115, 125]

# Calculate the mean sales revenue
mean_revenue = np.mean(sales_revenue)

# Calculate the median sales revenue
median_revenue = np.median(sales_revenue)

# Calculate the standard deviation of sales revenue
std_dev_revenue = np.std(sales_revenue)

print(f"Mean sales revenue: ${mean_revenue*100:.2f}")
print(f"Median sales revenue: ${median_revenue*100:.2f}")
print(f"Standard Deviation of sales revenue: ${std_dev_revenue*100:.2f}")
```

```
Mean sales revenue: $11735.71
Median sales revenue: $11750.00
Standard Deviation of sales revenue: $1242.47
```

2) A librarian records how many books were borrowed by different students in a week:

Books Borrowed : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Frequency (Number of Students): 3, 5, 8, 6, 7, 4, 3, 2, 1, 1

Compute the mean, median, mode, variance, and standard deviation of books borrowed.

```
import numpy as np
sales_revenue = [105, 98, 120, 130, 125, 110, 105, 115, 100, 140, 135, 120, 115, 125]

# Calculate the mean sales revenue
mean_revenue = np.mean(sales_revenue)

# Calculate the median sales revenue
median_revenue = np.median(sales_revenue)

# Calculate the standard deviation of sales revenue
std_dev_revenue = np.std(sales_revenue)

print(f"Mean sales revenue: ${mean_revenue*100:.2f}")
print(f"Median sales revenue: ${median_revenue*100:.2f}")
print(f"Standard Deviation of sales revenue: ${std_dev_revenue*100:.2f}")


books_borrowed = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
frequency = [3, 5, 8, 6, 7, 4, 3, 2, 1, 1]

# Combine books borrowed and frequencies into a single list for calculations
all_books = []
for i in range(len(books_borrowed)):
    all_books.extend([books_borrowed[i]] * frequency[i])

# Mean
mean = np.mean(all_books)

# Median
median = np.median(all_books)
```

```
# Mode
from collections import Counter
counts = Counter(all_books)
mode = counts.most_common(1)[0][0]

# Variance
variance = np.var(all_books)

# Standard Deviation
std_dev = np.std(all_books)

print("\nStatistics of Books Borrowed:")
print(f"Mean: {mean:.2f}")
print(f"Median: {median:.2f}")
print(f"Mode: {mode}")
print(f"Variance: {variance:.2f}")
print(f"Standard Deviation: {std_dev:.2f}")
```

```
Mean sales revenue: $11735.71
Median sales revenue: $11750.00
Standard Deviation of sales revenue: $1242.47

Statistics of Books Borrowed:
Mean: 4.40
Median: 4.00
Mode: 3
Variance: 4.79
Standard Deviation: 2.19
```

3) A highway patrol records the speed (in km/h) of vehicles over 10 intervals:

Speed Range (km/h): 40-50, 50-60, 60-70, 70-80, 80-90, 90-100, 100-110, 110-120, 120-130, 130-140

Number of Vehicles: 3, 5, 8, 10, 7, 6, 5, 3, 2, 1

Find the mean, median, mode, variance, and standard deviation of vehicle speeds.

```
import numpy as np
from collections import Counter

speed_ranges = [range(40, 51), range(50, 61), range(60, 71), range(70, 81),
                range(80, 91), range(90, 101), range(100, 111), range(110, 121),
                range(120, 131), range(130, 141)]
num_vehicles = [3, 5, 8, 10, 7, 6, 5, 3, 2, 1]

all_speeds = []
for i in range(len(speed_ranges)):
  for speed in speed_ranges[i]:
    all_speeds.extend([speed] * num_vehicles[i])

# Mean
mean_speed = np.mean(all_speeds)

# Median
median_speed = np.median(all_speeds)

# Mode
counts = Counter(all_speeds)
mode_speed = counts.most_common(1)[0][0]

# Variance
variance_speed = np.var(all_speeds)

# Standard Deviation
std_dev_speed = np.std(all_speeds)

print("\nStatistics of Vehicle Speeds:")
print(f"Mean: {mean_speed:.2f} km/h")
print(f"Median: {median_speed:.2f} km/h")
print(f"Mode: {mode_speed} km/h")
print(f"Variance: {variance_speed:.2f} (km/h)^2")
print(f"Standard Deviation: {std_dev_speed:.2f} km/h")
```

```
Statistics of Vehicle Speeds:
Mean: 82.00 km/h
Median: 79.00 km/h
Mode: 70 km/h
Variance: 491.00 (km/h)^2
Standard Deviation: 22.16 km/h
```

4) The data shows the Shoe Size and the IQ Score of 6 persons. The values are recorded below:

Person : 1 2 3 4 5

Shoe Size (US): 7 10 9 8 11

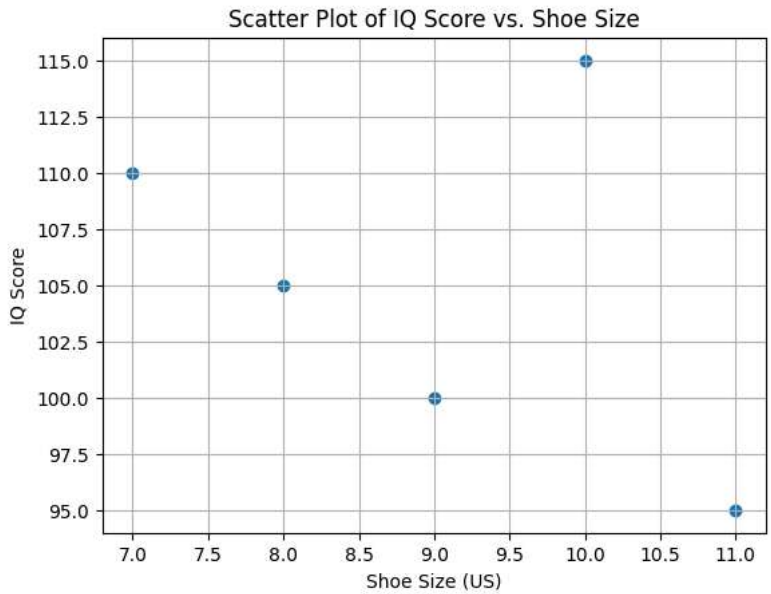
IQ Score : 110 115 100 105 95

(i) Plot a scatter diagram of IQ Score (y) against Shoe Size (x).

(ii) Calculate the correlation coefficient between the amount of Shoe Size and the IQ Score.

(iii) Interpret the correlation.

```
import matplotlib.pyplot as plt
import numpy as np

# Shoe sizes and IQ scores
shoe_sizes = [7, 10, 9, 8, 11]
iq_scores = [110, 115, 100, 105, 95]

# (i) Scatter plot
plt.scatter(shoe_sizes, iq_scores)
plt.xlabel("Shoe Size (US)")
plt.ylabel("IQ Score")
plt.title("Scatter Plot of IQ Score vs. Shoe Size")
plt.grid(True)
plt.show()

# (ii) Correlation coefficient
correlation_coefficient = np.corrcoef(shoe_sizes, iq_scores)[0, 1]
print(f"Correlation Coefficient: {correlation_coefficient:.2f}")

# (iii) Interpretation
if correlation_coefficient > 0.7:
    interpretation = "Strong positive correlation"
elif correlation_coefficient > 0.3:
    interpretation = "Moderate positive correlation"
elif correlation_coefficient > -0.3:
    interpretation = "Weak correlation"
elif correlation_coefficient > -0.7:
    interpretation = "Moderate negative correlation"
else:
    interpretation = "Strong negative correlation"

print(f"Interpretation: {interpretation}")
```




```
Correlation Coefficient: -0.40
Interpretation: Moderate negative correlation
```

5) The data shows the years of experience and the salary of 6 employees. The values are recorded below:

Employee: 1 2 3 4 5

Years of Experience: 1 3 5 7 10

Salary: 30,000 40,000 55,000, 70,000 90,000

(i) Plot a scatter diagram of salary (y) against years of experience (x).

(ii) Calculate the correlation coefficient between the years of experience and the salary.

(iii) Interpret the correlation. Does more years of experience generally lead to higher salaries?

```
import matplotlib.pyplot as plt
import numpy as np
# Years of experience and salaries
years_experience = [1, 3, 5, 7, 10]
salaries = [30000, 40000, 55000, 70000, 90000]

# (i) Scatter plot
plt.scatter(years_experience, salaries)
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Scatter Plot of Salary vs. Years of Experience")
plt.grid(True)
plt.show()

# (ii) Correlation coefficient
correlation_coefficient = np.corrcoef(years_experience, salaries)[0, 1]
print(f"Correlation Coefficient: {correlation_coefficient:.2f}")

# (iii) Interpretation
if correlation_coefficient > 0.7:
    interpretation = "Strong positive correlation"
elif correlation_coefficient > 0.3:
    interpretation = "Moderate positive correlation"
elif correlation_coefficient > -0.3:
    interpretation = "Weak correlation"
elif correlation_coefficient > -0.7:
    interpretation = "Moderate negative correlation"
else:
    interpretation = "Strong negative correlation"

print(f"Interpretation: {interpretation}")

if correlation_coefficient > 0:
  print("More years of experience generally lead to higher salaries.")
else:
  print("More years of experience do not generally lead to higher salaries.")
```

```
Correlation Coefficient: 1.00
Interpretation: Strong positive correlation
More years of experience generally lead to higher salaries
```

6) Calculate the correlation coefficient for the following heights (in inches) of fathers (X) and their sons (Y)

X:65 66 67 67 68 69 70 72

Y:61 68 65 68 72 72 69 71

```
import numpy as np
# Heights of fathers and sons
fathers_heights = [65, 66, 67, 67, 68, 69, 70, 72]
sons_heights = [61, 68, 65, 68, 72, 72, 69, 71]

# Calculate the correlation coefficient
correlation_coefficient = np.corrcoef(fathers_heights, sons_heights)[0, 1]

print(f"Correlation Coefficient: {correlation_coefficient:.2f}")
```

```
Correlation Coefficient: 0.70
```

7) Write a python code for Predicting Salary Based on Experience using lines of regression.

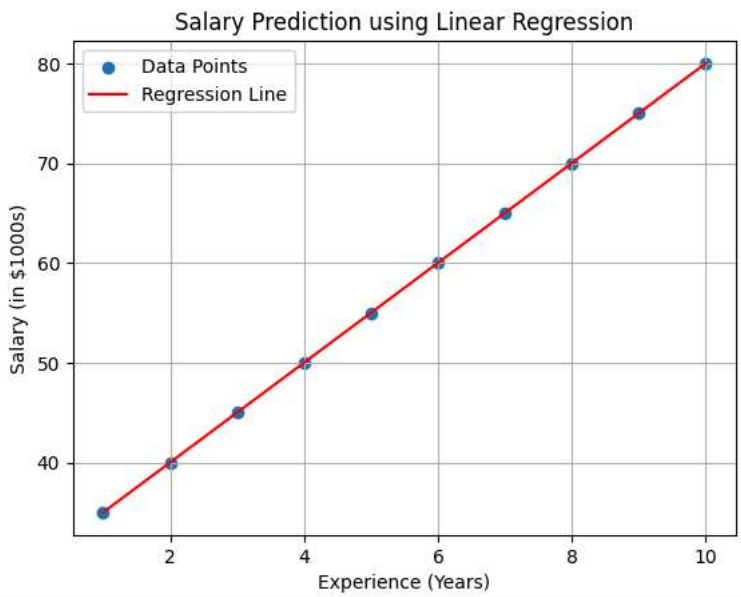
Experience (Years): 1 2 3 4 5 6 7 8 9 10

Salary (in $1000s): 35 40 45 50 55 60 65 70 75 80

```
import numpy as np
import matplotlib.pyplot as plt

# Experience (Years)
experience = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Salary (in $1000s)
salary = np.array([35, 40, 45, 50, 55, 60, 65, 70, 75, 80])

# Calculate the coefficients (slope and intercept) of the linear regression line
# using the least squares method.
n = len(experience)
x_mean = np.mean(experience)
y_mean = np.mean(salary)

numerator = np.sum((experience - x_mean) * (salary - y_mean))
denominator = np.sum((experience - x_mean)**2)
slope = numerator / denominator
intercept = y_mean - (slope * x_mean)
```

```
# Print the regression equation
print(f"Regression Equation: Salary = {slope:.2f} * Experience + {intercept:.2f}")


# Predict salary for a given experience
def predict_salary(exp):
    return slope * exp + intercept


# Example: Predict salary for 12 years of experience
predicted_salary_12 = predict_salary(12)
print(f"Predicted salary for 12 years of experience: ${predicted_salary_12 * 1000:.2f}")

# Plot the regression line and the data points
plt.scatter(experience, salary, label='Data Points')
plt.plot(experience, slope * experience + intercept, color='red', label='Regression Line')
plt.xlabel('Experience (Years)')
plt.ylabel('Salary (in $1000s)')
plt.title('Salary Prediction using Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

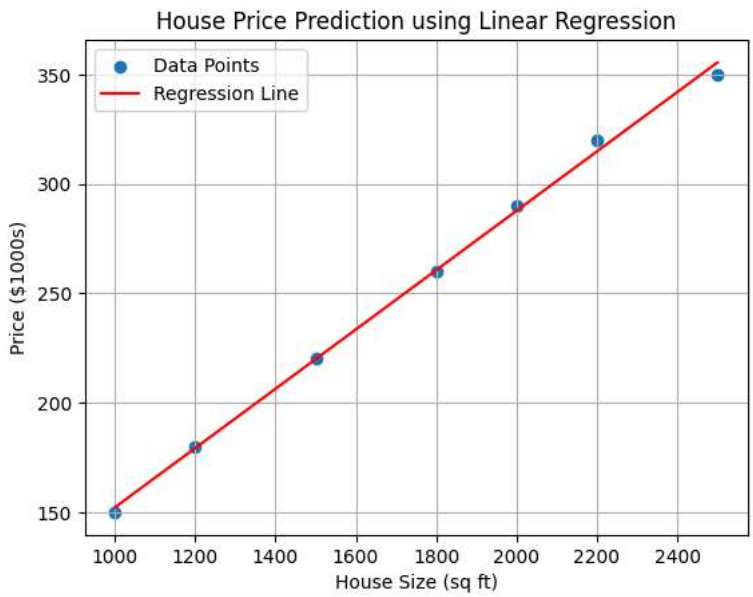
```
Regression Equation: Salary = 5.00 * Experience + 30.00
Predicted salary for 12 years of experience: $90000.00
```



8) Write a python code for Predicting House Price Based on Size using lines of regression.

- House Size (sq ft):1000 , 1200 , 1500 , 1800 , 2000 , 2200 , 2500
- Price ($1000s) : 150 , 180 ,220 , 260 , 290 , 320 , 350

```
import matplotlib.pyplot as plt
import numpy as np
# House size (sq ft)
house_size = np.array([1000, 1200, 1500, 1800, 2000, 2200, 2500])

# Price ($1000s)
price = np.array([150, 180, 220, 260, 290, 320, 350])

# Calculate the coefficients (slope and intercept) of the linear regression line
# using the least squares method.
n = len(house_size)
x_mean = np.mean(house_size)
y_mean = np.mean(price)

numerator = np.sum((house_size - x_mean) * (price - y_mean))
denominator = np.sum((house_size - x_mean)**2)
slope = numerator / denominator
intercept = y_mean - (slope * x_mean)

# Print the regression equation
```

```
print(f"Regression Equation: Price = {slope:.2f} * House Size + {intercept:.2f}")

# Predict price for a given house size
def predict_price(size):
    return slope * size + intercept

# Example: Predict price for a house size of 2300 sq ft
predicted_price_2300 = predict_price(2300)
print(f"Predicted price for a 2300 sq ft house: ${predicted_price_2300 * 1000:.2f}")

# Plot the regression line and the data points
plt.scatter(house_size, price, label='Data Points')
plt.plot(house_size, slope * house_size + intercept, color='red', label='Regression Line')
plt.xlabel('House Size (sq ft)')
plt.ylabel('Price ($1000s)')
plt.title('House Price Prediction using Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

```
Regression Equation: Price = 0.14 * House Size + 16.65
Predicted price for a 2300 sq ft house: $328365.85
```